

HDB Resale Price Predictor

Eugene Tan Boon Hong

Singapore Polytechnic Applied AI & Analytics

Overview

My project trains a regression model to predict HDB resale prices based on these flat characteristics: location, size, remaining lease, floor level, flat type, and flat model. The goal was not just to build a model, but to go through the full real world Machine Learning process; cleaning messy data, engineering useful features, comparing model approaches, and validating predictions against actual current market listings.

Data Source

- Dataset: HDB Resale Flat Prices (Jan 2017 onwards) — official transaction records published by data.gov.sg
https://data.gov.sg/datasets/d_8b84c4ee58e3cfcoeceod773c8ca6abc/view
- Size: 239,330 REAL transactions
- Columns: month, town, flat type, block, street name, storey range, floor area, flat model, lease commencement date, remaining lease, resale price

Data Cleaning & Feature Engineering

The government data was not analysis ready straight from the .csv file, Three columns needed transformation before they could be used.

Raw Column	Problem	Solution
remaining_lease	"61 years 04 months" (text)	Parsed to extract just the year count as an integer
storey_range	"10 TO 12" (text range)	Split and averaged into a single numeric floor estimate
month	"2017-01" (date string)	Converted to "months since Jan 2017" to capture market trend over time

town, flat_type, and flat_model (categorical/text fields) were one hot encoded to make them usable in my regression model

```
df["remaining_lease_years"] = df["remaining_lease"].str.split(" ").str[0].astype(int)

storeys = df["storey_range"].str.split(" TO ", expand=True).astype(int)
df["floor_avg"] = (storeys[0] + storeys[1]) / 2

print(df[["remaining_lease", "remaining_lease_years", "storey_range", "floor_avg"]].head())
```

✓ 0.4s

	remaining_lease	remaining_lease_years	storey_range	floor_avg
0	61 years 04 months	61	10 TO 12	11.0
1	60 years 07 months	60	01 TO 03	2.0
2	62 years 05 months	62	01 TO 03	2.0
3	62 years 01 month	62	04 TO 06	5.0
4	62 years 05 months	62	01 TO 03	2.0

Model Iterations

Rather than accepting the first result, each change was tested in isolation to measure its actual impact:

Version	Type of Model	Mean Absolute Error	% of Average Price (\$534,296)
V1	Linear Regression	\$52,760	9.87%
V2	Random Forest	\$26,811	5.01%

Linear Regression Model V1

```
lr_model = LinearRegression()
lr_model.fit(X_train, y_train)
lr_predictions = lr_model.predict(X_test)
print("Linear Regression error:", mean_absolute_error(y_test, lr_predictions))
```

✓ 0.4s

Linear Regression error: 52760.87851391757

Random Forest Model V2

```
rf_model = RandomForestRegressor(n_estimators=100, random_state=42)
rf_model.fit(X_train, y_train)
rf_predictions = rf_model.predict(X_test)
print("Random Forest error:", mean_absolute_error(y_test, rf_predictions))
```

✓ 2m 22.4s

Random Forest error: 26811.401815377176

Key Findings

- Switching from a Linear Regression to Random Forest Model nearly halved the prediction error. From \$52,760 to \$26,811, a reduction of about 49%
- As a percentage of the average resale price (\$534,296), this moved the model from roughly 9.87% off to 5.01% off, a meaningfully more accurate result.
- The improvement comes from how each model handles relationships in the data: Linear Regression assumes every feature adds up in a straight line, while Random Forest can capture other factors in its interactions; for example, floor area may matter more in some towns than others, a pattern a purely linear model could not represent
- The trade-off is, the Random Forest Model took a lot longer to train (2m 22s vs 0.4s), reflecting the added computational cost of building 100 decision trees instead of fitting a single linear equation.

Real-World Validation

To check the model beyond the test set and against real world properties, I pulled live PropertyGuru listings and compared the model's prediction against the actual asking price.

Listing	Town	Type	Size	Lease	Asking	Predicted
319 AMK Ave 1	Ang Mo Kio	3 Rm	73sqm	50yr	\$440,000	\$434,234
649 Hougang Ave 8	Hougang	5 Rm	120sqm	60yr	\$808,000	\$778,804
588D AMK St 52	Ang Mo Kio	5 Rm	112sqm	84yr	\$1,180,000	\$936,113

Note that the asking price is the price of what the seller WANTS, PropertyGuru did not state the exact floor of the listing, hence I had to gauge it based on the listing descriptions of "High Floor" etc.

Listing Screenshots

319 Ang Mo Kio Avenue 1

319 Ang Mo Kio Avenue 1, Ang Mo Kio / Bishan / Thomson

S\$ 440,000 Negotiable



2
Beds



2
Baths



786
sqft



S\$ 559.80
psf

649 Hougang Avenue 8

649 Hougang Avenue 8, Hougang / Punggol / Sengkang

S\$ 808,000 Negotiable



3
Beds



2
Baths



1,292
sqft



S\$ 625.39
psf

588D Ang Mo Kio Street 52

588D Ang Mo Kio Street 52, Ang Mo Kio / Bishan / Thomson

S\$ 1,180,000



3
Beds



2
Baths



1,205
sqft



S\$ 979.25
psf

Prediction Code

The `predict_price()` function used to generate each prediction above:

```
def predict_price(town, flat_type, flat_model, floor_area_sqm, remaining_lease_years, floor_avg, months_since_start):
    input_data = pd.DataFrame(columns=X.columns)
    input_data.loc[0] = 0

    input_data.loc[0, "floor_area_sqm"] = floor_area_sqm
    input_data.loc[0, "remaining_lease_years"] = remaining_lease_years
    input_data.loc[0, "floor_avg"] = floor_avg
    input_data.loc[0, "months_since_start"] = months_since_start

    town_col = f"town_{town}"
    flat_type_col = f"flat_type_{flat_type}"
    flat_model_col = f"flat_model_{flat_model}"

    if town_col in input_data.columns:
        input_data.loc[0, town_col] = 1
    if flat_type_col in input_data.columns:
        input_data.loc[0, flat_type_col] = 1
    if flat_model_col in input_data.columns:
        input_data.loc[0, flat_model_col] = 1

    predicted = rf_model.predict(input_data)[0]
    return f"${round(predicted):,}"

def months_since_jan_2017(year, month):
    return (year - 2017) * 12 + month
```

✓ 0.0s

Tech Stack

- Python: pandas, scikit-learn
- Environment: Jupyter Notebook
- Models tested: Linear Regression, Random Forest Regressor

What I would Improve Next

- Add distance to MRT and distance to nearby necessities like malls & schools as features (These are likely significant price drivers that were not captured)
- Build a small interactive input (Streamlit or similar) instead of manually calling the prediction function which could get repetitive
- Expand real world validation to a larger sample of 10+ real PropertyGuru listings for a proper accuracy check, rather than a handful of spot checks

Key Takeaway

The biggest lesson from this project was not a specific line of code but learning to treat model improvement as an experiment; change one thing, measure the effect, and understand why it did or did not help, rather than tweaking blindly until a number looks better.